

Smart Fisherman Assistance and Safety System (SFASS)

Project Creator & Lead Developer: Thirupathi Aashrith

Domain: Internet of Things (IoT), Embedded Systems, Marine Safety & Telemetry

1. Project Purpose & Background

Fishing remains one of the most hazardous occupations globally due to unpredictable weather, extreme sea conditions, capsizing risks, and health emergencies far from shore. Traditional marine communication gear is often expensive or inaccessible for small-scale vessel operators. The **Smart Fisherman Assistance and Safety System (SFASS)** was conceptualized and developed by **Thirupathi Aashrith** to provide an accessible, real-time safety ecosystem built on the **ESP32 microcontroller**. It continuously monitors both environmental parameters and the vessel operator's vitals, autonomously dispatching alerts and logging telemetry data to the cloud.

2. Real-Life Applications & Impact

- **Instant Capsize Detection & Emergency Dispatch:** Automatically detects when a boat overturns ($>90^\circ$ tilt angle) using an accelerometer and sends instant emergency notifications with live GPS location markers via Telegram.
- **Manual SOS Safety Net:** Includes a tactile SOS button for fishermen in distress (e.g., engine breakdown, medical emergencies, or maritime safety threats).
- **Vessel Health & Weather Monitoring:** Tracks barometric pressure and atmospheric shifts to help predict localized storms or sudden weather drops before they become hazardous.
- **Biometric Operator Tracking:** Constantly measures operator heart rates to detect severe stress, exhaustion, or cardiac distress during extended shifts at sea.
- **Marine Water Quality Analysis:** Measures water temperature and Total Dissolved Solids (TDS) for oceanographic logging and fishing spot suitability.
- **Cloud Telemetry (ThingSpeak):** Streams vital data to cloud dashboards for central monitoring by coast guards, marine harbor masters, or family members.

3. Hardware Architecture & Components

Component	Hardware Model	Purpose & Role in SFASS
Microcontroller	ESP32 DevKit V1	Core processing unit with dual-core architecture and Wi-Fi capability.

GPS Module	NEO-6M	Tracks geolocation coordinates (Latitude/Longitude) for real-time positioning.
Tilt/Inertial Sensor	ADXL345	3-axis MEMS accelerometer to measure boat tilt angles and detect capsize events.
Water Temp Sensor	DS18B20	Waterproof 1-Wire temperature sensor for marine thermal analysis.
Pressure Sensor	BMP180 / BMP085	Barometric pressure & altitude sensor for storm and weather predictions.
Water Purity Sensor	TDS Sensor Module	Analog sensor measuring Total Dissolved Solids in water (ppm).
Heart Rate Sensor	Analog Pulse Sensor	PPG optical pulse sensor for operator heart rate monitoring (BPM).
Audio Feedback	Passive Buzzer	Generates custom tone frequencies for SOS, capsize alerts, and system feedback.
Emergency Input	Push Button	Connected with internal pull-up logic to trigger manual SOS alerts.



4. ESP32 Circuit Wiring & Pin Mapping

Ensure all components are wired according to the pin mapping below before running the firmware:

Component	ESP32 Pin	Interface / Protocol	Connection Details
DS18B20 Data	GPIO 27	OneWire	Requires a 4.7kΩ pull-up resistor between 3.3V and Data.
SOS Push Button	GPIO 33	Digital Input	One leg to GPIO 33, other leg to GND .
TDS Sensor	GPIO 35	Analog Input (ADC1)	Signal pin to GPIO 35.
Pulse Sensor	GPIO 34	Analog Input (ADC1)	Signal pin to GPIO 34.
Passive Buzzer	GPIO 5	PWM / Tone	Positive pin to GPIO 5, negative to GND .
NEO-6M GPS (TX)	GPIO 16	UART2 (RX2)	Connect GPS TX to ESP32 GPIO 16 .
NEO-6M GPS (RX)	GPIO 17	UART2 (TX2)	Connect GPS RX to ESP32 GPIO 17 .
ADXL345 & BMP180	GPIO 21 (SDA)	I2C Bus	Both sensors share the SDA line.
ADXL345 & BMP180	GPIO 22 (SCL)	I2C Bus	Both sensors share the SCL line.



5. Software Architecture & Dependencies

The system firmware handles concurrent sensor sampling, signal filtering, emergency interrupt checks, and cloud sync routines.

Required Arduino IDE Libraries

Install the following libraries via the Arduino Library Manager (**Sketch** $\rightarrow$ **Include Library** $\rightarrow$ **Manage Libraries**):

- WiFi & WiFiClientSecure (ESP32 Built-in)
- ThingSpeak by MathWorks
- Adafruit BMP085 Library by Adafruit
- Adafruit ADXL345 & Adafruit Unified Sensor by Adafruit
- OneWire by Paul Stoffregen
- DallasTemperature by Miles Burton
- TinyGPS++ by Mikal Hart
- UniversalTelegramBot by Brian Lough
- ArduinoJson by Benoit Blanchon



6. Repository File Structure

Organize your GitHub repository as follows:

```

Plaintext
Smart-Fisherman-Assistance-and-Safety-System
├── src
│   └── main.ino          # Complete ESP32 firmware source file
├── docs
│   ├── circuit-diagram.png
│   └── hardware-setup.jpg
├── README.md            # Project documentation
└── LICENSE               # MIT or Open Source License

```



7. Step-by-Step Instructions to Upload to GitHub

- 1. Create a GitHub Repository:**
 - Log into [GitHub](#) and click **New Repository**.
 - Repository Name: Smart-Fisherman-Assistance-and-Safety-System.
 - Check **Add a README file**.
- 2. Add Source Files:**
 - Create a folder named `src/` and place your `.ino` source file inside it.
 - Paste this markdown text directly into your repository's root `README.md` file.

3. Include Visual Assets:

- Upload photos of your hardware prototype or schematic to a docs/ folder and link them directly in the README for maximum portfolio impact.



8. Future Developments

- **Long-Range LoRa Integration:** Supplementing Wi-Fi with LoRa communication to transmit SOS telemetry when the vessel is deep offshore beyond cellular coverage.
- **On-board Solar Harvesting:** Integrating a solar management circuit to ensure uninterrupted operation during extended fishing journeys.
- **Predictive Edge AI:** Implementing lightweight ML models on the ESP32 to predict stormy conditions based on 6-hour barometric pressure trends.

Author & Developer Credits:

Lead Developer: Thirupathi Aashrith

Project Title: Smart Fisherman Assistance and Safety System (SFASS)